

שיעור 8 - JAVA

נושא 1: הרכב אובייקטים והשמת מצביעים בזיכרון (Object Composition & Pointer Assignment)

הסבר מקוצר:

הרכב אובייקטים (Composition) הוא קשר שבו מחלקה אחת מכילה תכונה שהיא בעצמה אובייקט ממחלקה אחרת. בעת יצירת האובייקט, הקונסטרקטור מקבל את כתובת הזיכרון של האובייקט החיצוני ומבצע השמת מצביעים המחברת ביניהם.

הסבר מורחב:

בפרויקט הבנק שנלמד, המחלקה CashAccount (חשבון ע"ש) אינה שומרת את פרטי הבעלים כמשתנים פשוטים (כמו שם או מספר לקוח) מפני שבעל חשבון מכיל מידע רב ומורכב. במקום זאת, היא מחזיקה תכונה שהיא אובייקט מסוג AccountOwner. כאשר התוכנית הראשית (main) יוצרת חשבון חדש, היא מעבירה לקונסטרקטור של CashAccount את כתובת האובייקט של הבעלים. הקונסטרקטור מבצע השמת מצביעים (this.owner = owner), המעתיקה את הכתובת של אובייקט הבעלים אל תוך התכונה הפנימית של החשבון. כתוצאה מכך, נוצר מצב שבו החשבון מצביע על אובייקט הבעלים בזיכרון. אם נחזיק בהפניה לאותו אובייקט בעלים בתוכנית הראשית ונשנה את אחד משדותיו (למשל, שינוי שם המשפחה), השינוי ישתקף באופן מיידי גם בתוך אובייקט החשבון, מכיוון ששני המשתנים מצביעים פיזית על אותו עצם בזיכרון.

משפטי מפתח והגדרות למבחן:

- **הרכב אובייקטים (Composition):** דפוס עיצוב שבו מחלקה אחת מכילה שדות שהם הפניות (References) לאובייקטים ממחלקות אחרות, מה שמאפשר ייצוג ישויות מורכבות בעלות נתונים רבים.
- **השמת מצביעים (Pointer Assignment):** פעולת השמה שאינה מעתיקה את ערכי האובייקט או מייצרת לו עותק חדש, אלא מעתיקה את הכתובת שלו בזיכרון, ובכך מאפשרת למספר משתנים להצביע על אותו האובייקט.
- **שיתוף משאבים בזיכרון:** כאשר אובייקט פנימי מוצבע על ידי גורם חיצוני ופנימי בו-זמנית, כל עדכון של האובייקט דרך משתנה אחד ישפיע ישירות על הנתונים הנצפים דרך המשתנה השני.

דוגמאות:

- בתוכנית הבנק נוצר אובייקט בעלים עבור הלקוח "רוס גלר" והועבר לקונסטרקטור של החשבון acc1. בסוף השיעור הקודם נשלפה הפניית הבעלים לתוך משתנה ב-main ושונה שם המשפחה ל"כהן". בהדפסת החשבון acc1 הוכח כי שם המשפחה השתנה ל"כהן" גם מנקודת מבטו של החשבון.

שאלות הבנה:

1. מה ההבדל בין השמת משתנה פשוט (כמו int) לבין השמת משתנה מסוג אובייקט בקונסטרקטור של מחלקה?
2. אם נמחק או נאפס את המשתנה המקורי שהצביע על הבעלים בתוך מתודת ה-main, האם המידע על הבעלים יימחק גם מתוך אובייקט החשבון? נמקו.
3. מדוע מבחינת הארכיטקטורה של הקוד עדיף לייצג את בעל החשבון כאובייקט עצמאי ולא לשמור את השם?

והאימייל שלו כתכונות פשוטות ישירות במחלקת CashAccount?

נושא 2: דריסת המתודה toString() ושרשור אובייקטים

במחרוזות

הסבר מקוצר:

המתודה toString() מחזירה ייצוג מחרוזתי מעוצב של האובייקט וערכי תכונותיו. Java מפעילה את המתודה הזו באופן סמוי ואוטומטי בכל פעם שאובייקט נשלח להדפסה או משרשר בתוך מחרוזת טקסט.

הסבר מורחב:

בכל מחלקה שנבנה ב-Java, ניתן להגדיר מתודה ציבורית המחזירה String ונקראת בדיוק בשם toString(). אם לא נממש מתודה זו וננסה להדפיס את האובייקט, Java תשתמש במימוש ברירת המחדל הגנרי שלה, אשר מדפיס את שם המחלקה, סימן שטרודל (@), ומחרוזת המייצגת את כתובת הזיכרון שלו. כאשר אנו דורסים את המתודה, אנו יכולים לעצב את המחרוזת כרצוננו ולשלב בה משתנים מסוגים שונים (כמו int או double) ללא צורך בהמרה ידנית, שכן Java יודעת לשרשר טיפוסים אלו אוטומטית לתוך מחרוזת. החזק של המנגנון בא לידי ביטוי בהרכב אובייקטים: כאשר אנו כותבים בתוך ה-toString() של מחלקת החשבון את הפקודה this.owner + "...", אנו בעצם משרשרים אובייקט לתוך מחרוזת. Java מזהה זאת, קוראת באופן אוטומטי ל-toString() של אובייקט הבעלים, ומציבה את מחרוזת הפרטים שלו בתוך המחרוזת הכללית של החשבון.

משפטי מפתח והגדרות למבחן:

- **מתודת toString():** מתודה בעלת חתימה קבועה (public String toString()) המשמשת להפקת תיאור טקסטואלי של האובייקט ומאפייניו.
- **הפעלה סמויה (Implicit Invocation):** תכונה של שפת Java המפעילה אוטומטית את toString() של אובייקט בכל פעם שהוא מועבר ישירות לפקודת הדפסה (System.out.print) או כאשר הוא משורשר למחרוזת אחרת באמצעות אופרטור +.
- **שרשור אובייקטים מורכב:** מנגנון המאפשר ל-toString() של מחלקה אחת להסתמך על ה-toString() של מחלקה פנימית מוכלת, ובכך ליצור פלט היררכי מסודר בקלות.

דוגמאות:

- במחלקה AccountOwner נבנתה מחרוזת מעוצבת עם כותרות קריאות עבור תעודת זהות, שם, אימייל וטלפון. במחלקת CashAccount, ה-toString() משלבת את מספר החשבון והיתרה, ובסופה משרשרת את this.owner, מה שמביא להדפסה משולבת ומלאה של כל נתוני החשבון והלקוח בריצה אחת.

שאלות הבנה:

1. מה יקרה לקוד בזמן קומפילציה או בזמן ריצה אם נגדיר את המתודה בשם toString() (באותיות קטנות בלבד) וננסה להדפיס את האובייקט ישירות?
2. כיצד מתנהגת Java בעת שרשור משתנים פרימיטיביים (כמו int או float) לתוך מחרוזת, ומה ההבדל בינה לבין שפות תכנות אחרות כדוגמת פייתון בהקשר זה?
3. תארו מה יודפס למסך אם נבצע הדפסה לאובייקט חשבון שלא מוגדרת בו המתודה toString(), אך באובייקט הבעלים שלו (owner) המתודה כן מוגדרת וממומשת כראוי.

נושא 3: השוואת אובייקטים ומימוש מתודת isEqual מותאמת

הסבר מקוצר:

אופרטור השוויון == משווה בין כתובות הזיכרון של אובייקטים ולא בין הערכים הפנימיים שלהם. כדי לבצע השוואת תוכן לוגית, יש לממש מתודה ייעודית (כגון isEqual) המקבלת אובייקט כפרמטר ומגדירה קריטריון השוואה מותאם.

הסבר מורחב:

בעוד שטיפוסים פרימיטיביים כמו int או double מושוים על פי ערכם הפיזי באמצעות ==, השוואת אובייקטים באמצעות אופרטור זה תבדוק אך ורק האם שני המשתנים מצביעים על אותו מקום בדיוק בזיכרון. כדי לפתור זאת, מייצרים מתודה (בשלב זה בקורס נקראה isEqual) שמקבלת אובייקט אחר מאותו הסוג, לו נהוג לקרוא other.

המתכנת נדרש לקבוע את החוקים העסקיים של ההשוואה: במקרה של חשבון בנק, מספיק לוגית להשוות רק את מספר החשבון (this.number == other.number) מכיוון שלא ייתכנו שני חשבונות בעלי אותו מספר. לעומת זאת, אם נרצה השוואה מלאה של כל התכונות, נשתמש באופרטור וגם (&&) המשווה את כל השדות. במימוש מורכב, חובה לזכור ששדות שהם בעצמם אובייקטים (כמו מחרוזות לקבלת שם, או אובייקט הבעלים הפנימי) לא ישוו ב-== אלא באמצעות קריאה למתודת ההשוואה שלהם (equals) המובנית של מחרוזות או isEqual של הבעלים.

משפטי מפתח והגדרות למבחן:

- **שוויון כתובות (Reference Equality):** מצב שבו אופרטור == מחזיר true אך ורק אם שני המשתנים מחזיקים את אותה כתובת זיכרון ומפנים לאותו אובייקט פיזי.
- **שוויון תוכן (Logical Equality):** שוויון המוגדר על ידי המתכנת, שבו אובייקטים שונים לחלוטין בזיכרון נחשבים "שווים" מכיוון שהערכים של התכונות שלהם (או של שדה מפתח ייחודי) זהים.
- **Short-circuit evaluation (אופרטור &&):** תכונה של שפת Java שבה אם הביטוי הראשון בתוך תנאי מורכב של "וגם" מחזיר false, המערכת לא תמשיך להעריך את שאר חלקי הביטוי ותעצור מייד, מה שמגביר את יעילות הריצה.
- **השוואת מחרוזות ב-Java:** מחרוזת (String) היא אובייקט ב-Java ולכן השוואה בין שתי מחרוזות מתבצעת תמיד באמצעות המתודה equals או equalsIgnoreCase() ולא באמצעות אופרטור ==.

דוגמאות:

- במימוש המלא של isEqual במחלקת AccountOwner, הושוו שדות ה-String השונים באמצעות מתודת equals המובנית של מחרוזות: (this.firstName.equals(other.firstName)).
- במימוש המורכב של מחלקת החשבון, בוצעה השוואה משולבת: return this.number == other.number && this.owner.isEqual(other.owner);.

שאלות הבנה:

1. נניח שקיימים שני חשבונות בנק שונים בזיכרון, אך לשניהם יש את אותו מספר חשבון בדיוק. מה תחזיר המתודה acc1.isEqual(acc2) על פי המימוש הבסיסי, ומה יחזיר הביטוי acc1 == acc2?
2. מדוע שימוש באופרטור == לצורך השוואת שמות לקוחות (מסוג String) בתוך מתודת ה-isEqual עלול להחזיר false גם אם לשני האנשים קוראים "רוס"?
3. כיצד מנגנון ה-Short-circuit בא לידי ביטוי כאשר אנו מפעילים את המימוש המורכב של isEqual על שני

חשבונות שהמספר שלהם שונה לחלוטין? אילו בדיקות ייחסכו?

נושא 4: ניהול מערך אובייקטים במחלקה מחזיקה (Managing an Object Array in a Containing Class)

הסבר מקוצר:

מחלקה מורכבת (כגון Bank) יכולה להכיל מערך של אובייקטים כדי לנהל אוסף נתונים. מכיוון שמערך ב-Java הוא בעל גודל קבוע וסטטי, יש לנהל ידנית מונה המעלה על כמות האיברים שהוכנסו בפועל כדי למנוע חריגות ושגיאות זיכרון.

הסבר מורחב:

כאשר יוצרים מחלקה כמו Bank שתפקידה לנהל חשבונות רבים, מגדירים בתוכה תכונה שהיא מערך של אובייקטים: `private CashAccount[] accounts`; . מערך ב-Java הוא אובייקט בעל גודל קבוע אשר נקבע בעת ההקצאה שלו בקונסטרקטור (`new CashAccount[size]`) ולא ניתן להגדלה או הקטנה באופן דינמי. עם יצירת המערך, כל התאים שלו מאותחלים כברירת מחדל בערך `null`. כדי לנהל את הכנסת החשבונות בצורה מבוקרת ורציפה, חובה להחזיק משתנה מונה פנימי, למשל `count`, שמתחיל מאפס. משתנה זה משרת שתי מטרות קריטיות:

1. **במתודת ההוספה (`addAccount`):** הוא משמש כאינדקס לתא הפנוי הבא במערך. לפני ההשמה, אנו מוודאים שקיימים תאים פנויים במערך (`count < size`). אם התנאי מתקיים, האובייקט מוכנס לתא ה-`count` והמונה מקודם ב-1.
2. **במתודות `toString` (חיפוש):** הלולאות אינן רצות עד סוף המערך הפיזי (`size`), אלא רק עד האינדקס `count`. ריצה מעבר ל-`count` תביא לניסיון גישה לתאים שערכם `null`, וכל ניסיון להפעיל עליהם פונקציה (כמו `getNumber()`) יגרום מייד לזריקת חריגה חמורה מסוג `NullPointerException` ולקריסת התוכנית.

משפטי מפתח והגדרות למבחן:

- **מערך אובייקטים (`Array of Objects`):** מבנה נתונים בגודל קבוע שבו כל תא אינו מכיל את האובייקט עצמו, אלא משתנה מצביע השומר את כתובת הזיכרון של האובייקט.
- **משתנה מונה (`count`):** תכונה פנימית חיונית במחלקה המחזיקה מערך, המאפשרת מעקב מדויק אחר כמות האובייקטים שהוכנסו בפועל, הגנה מפני גלישה מהמערך ומניעת לולאות חיפוש מיותרות אחר תאים פנויים.
- **חריגת `NPE` (`NullPointerException`):** שגיאת זמן ריצה שכיחה המתרחשת כאשר קוד מנסה להפעיל מתודה או לגשת לתכונה של משתנה הפניה (`Reference`) שאינו מצביע על שום אובייקט בזיכרון ומכיל את הערך `null`.
- **החזרת ערך `null` בחיפוש:** פרקטיקה תכנותית (בהיעדר שימוש בחריגות) שבה פונקציית חיפוש (כמו `getAccount`) סורקת את המערך ואם לא מצאה את האובייקט המבוקש, היא מחזירה `null` כדי לאותת לקורא שהחיפוש נכשל.

דוגמאות:

- במתודה `getAccount` המקבלת מספר חשבון, הלולאה סורקת את המערך מ-0 ועד `count`. בכל איטרציה נבדק התנאי: `if (accounts[i].getNumber() == accountNumber)`. אם נמצא החשבון הוא מוחזר באופן מיידי. אם הלולאה מסתיימת ללא שום התאמה, התוכנית יוצאת מגבולות הלולאה ומבצעת `return null`;

שאלות הבנה:

1. מה יקרה לתוכנית בזמן ריצה אם ננסה להוסיף חשבון 11 לבנק שהוגדר מראש עם size שווה 10, וכיצד הקוד שכתבנו בשיעור מונע קריסה פיזית של התוכנית?
2. מדוע פונקציית החיפוש `getAccount` אינה יכולה להחזיר פשוט את הערך 0 או משתנה פרימיטיבי אחר כאשר החשבון לא נמצא במערך?
3. תארו דרך חלופית (רעיונית) שהועלתה בשיעור להתמודדות עם מצב שבו המערך מתמלא לחלוטין אך אנו עדיין רוצים לאפשר למשתמש להוסיף חשבונות חדשים לבנק.

טבלה מסכמת של הנושאים

נושא	הגדרות והרחבות מסכמות
הרכב אובייקטים והשמת מצביעים	קישור בין ישויות שונות בתוכנה על ידי הפיכת אובייקט אחד לתכונה בתוך אובייקט אחר . השמת המצביעים בקונסטרקטור חוסכת זיכרון ויוצרת סנכרון ישיר בין משתנים שונים המפנים לאותו המיקום בזיכרון .
דריסת המתודה toString()	יצירת מימוש מותאם אישית לפקודת ההדפסה של האובייקט . Java תומכת בהפעלה סמויה ומאפשרת שרשור היררכי אוטומטי של אובייקטים פנימיים כדי לייצר פלט טקסטואלי מורכב וקריא .
השוואת אובייקטים ו-isEqual	הבחנה קריטית בין שוויון כתובות בזיכרון (==) לבין שוויון תוכן לוגי שנקבע על ידי המתכנת . השוואת תכונות מורכבות מחייבת שימוש ב-equals עבור מחרוזות ובדיקות מקוננות עבור אובייקטים פנימיים .
ניהול מערך אובייקטים	אחסון קבוצה של אובייקטים במערך סטטי בתוך מחלקה המנהלת אותם . עבודה נכונה מחייבת שימוש במונה count כדי לשמור על רציפות המערך ולמנוע חריגות מסוג NullPointerException בעת ריצה על תאים ריקים .

סיכום פקודות ופונקציות (Java)

שם הפקודה/פונקציה	כיצד להשתמש	מה היא עושה	דוגמה קצרה
public String (<code>toString</code>)	הגדרה בתוך המחלקה כמתודה ציבורית המחזירה מחרוזת .	דורסת את ייצוג ברירת המחדל של האובייקט ומחזירה טקסט מובנה המייצג את נתוניו .	<pre>public String toString() { return "No: " + ;this.number</pre>
equals (של מחרוזת)	הפעלה על גבי משתנה מסוג String וקבלת מחרוזת אחרת כפרמטר.	משווה בין התוכן הלוגי של שתי מחרוזות (תו אחר תו) ומחזירה ערך בוליאני .	<pre>if (this.firstName.equa ((!s(other.firstName</pre>
equalsIgnoreCase (	הפעלה על גבי משתנה מסוג String וקבלת מחרוזת כפרמטר.	משווה בין תכני שתי מחרוזות תוך התעלמות מהבדלים בין אותיות גדולות לקטנות (Case (Insensitive).	<pre>this.name.equalsIgnoreCase(other.nam oreCase(other.nam (e</pre>
הקצאת מערך אובייקטים	שימוש במילת המפתח <code>new</code> , שם המחלקה וסוגריים מרובעים עם הגודל הרצוי .	מקצה מקום בזיכרון עבור מערך בגודל קבוע שיכול להכיל הפניות לאובייקטים מהסוג שהוגדר .	<pre>this.accounts = new ;[CashAccount[10</pre>

נספח: תשובות לשאלות ההבנה

נושא 1: הרכב אובייקטים והשמת מצביעים בזיכרון

- השמת משתנה פרימיטיבי (כגון `int`) מעתיקה באופן מלא את הערך המספרי לתא זיכרון חדש ועצמאי . לעומת זאת, השמת אובייקט מעתיקה אך ורק את כתובת הזיכרון שלו (המצביע) . כתוצאה מכך, לא נוצר אובייקט חדש, אלא שני המשתנים (החיצוני והתכונה הפנימית של המחלקה) חולקים גישה לאותו העצם בדיוק.
- לא, המידע לא יימחק מהזיכרון. למרות שהמשתנה ב-`main` כבר אינו מצביע עליו, אובייקט החשבון (`CashAccount`) עדיין מחזיק בתכונה הפנימית שלו (`this.owner`) מצביע פעיל ותקין המפנה לכתובת הזיכרון של אותו הבעלים . רכיב פנימי הזיכרון של Java (ה-`Garbage Collector`) מוחק אובייקטים רק כאשר אין אף משתנה בתוכנית כולה שמצביע עליהם.
- הפרדה זו שומרת על עקרון המודולריות והתכנות הנכון. בעל חשבון הוא ישות עצמאית בעלת מאפיינים רבים (תעודת זהות, אימייל, טלפון ועוד). אם נציב את כולם בתוך מחלקת החשבון, ננפח את המחלקה

בפרטים שאינם קשורים לניהול החשבון הפיננסי עצמו (כמו יתרה ופעולות הפקדה ומשיכה) . בנוסף, הרכב אובייקטים מאפשר לאותו לקוח (אובייקט בעלים יחיד) להיות משויך למספר חשבונות בנק שונים במקביל מבלי לשכפל את המידע שלו.

נושא 2: דריסת המתודה toString() ושרשור אובייקטים במחרוזות

1. שם המתודה ב-Java רגיש לאותיות גדולות וקטנות (Case Sensitive). אם נכתוב toString(), הקומפיילר או מערכת הריצה לא יזהו את המתודה כדריסה של מתודת ה-toString() הרשמית של השפה. בזמן הדפסת האובייקט, Java תתעלם מהפונקציה שכתבנו ותפעיל את מתודת ברירת המחדל הגנרית, והפלט שיוצג יהיה שם המחלקה וכתובת הזיכרון המשונה (למשל CashAccount@1a2b3c).
2. Java מאפשרת לשרשר טיפוסים פרימיטיביים ישירות לתוך מחרוזת טקסט באמצעות אופרטור + ומבצעת מאחורי הקלעים המרה אוטומטית לטקסט מבלי שהמתכנת ידרש לכתוב קוד מיוחד . בשפות כמו פייתון, ניסיון לשרשר מספר ישירות למחרוזת ללא המרה ידנית ומפורשת (באמצעות פונקציות כמו str()) יגרום לשגיאת טיפוס (Type Error) וכישליל את התוכנית.
3. בעת הדפסת אובייקט החשבון, Java תפעיל את ה-toString() של החשבון (אם מומשה) . מכיוון שבתוך ה-toString() של החשבון מופיע שרשור של אובייקט הבעלים (Java, this.owner), תזהה זאת ותפעיל באופן מקונן וסמוי את ה-toString() המעוצב של מחלקת הבעלים . כתוצאה מכך, נקבל פלט משולב ויפה שבו מוצגים פרטי החשבון, ובתוכם שזורים כל נתוני הלקוח הקריאים (תעודת זהות, שם וכו') .

נושא 3: השוואת אובייקטים ומימוש מתודת isEqual מותאמת

1. על פי המימוש הבסיסי שבוחן רק את מספר החשבון, המתודה acc1.isEqual(acc2) תבצע השוואה בין מספרי החשבון הפיזיים, ומכיוון שהם זהים היא תחזיר true. לעומת זאת, הביטוי acc1 == acc2 ישווה את כתובות הזיכרון של שני האובייקטים, ומכיוון שמדובר בשני עצמים שונים שנמצאים במקומות נפרדים בזיכרון, הוא יחזיר false .
2. מחרוזות ב-Java הן אובייקטים לכל דבר ועניין, ומשתנים מסוג String מחזיקים בתוכם כתובות זיכרון המצביעות על מקום הטקסט ולא על האותיות עצמן. שימוש באופרטור == ישווה את הכתובות של שתי המחרוזות בזיכרון. אם שתי המחרוזות נוצרו בנפרד, הן עשויות לשבת בכתובות שונות, ולכן אופרטור == יחזיר false למרות שתוכן הלוגי ("רוס") זהה לחלוטין. לשם כך חובה להשתמש ב-equals().
3. בזכות מנגנון ה-Short-circuit evaluation, אופרטור && מעריך את התנאים משמאל לימין. הבדיקה הראשונה היא השוואת מספרי החשבון פרימיטיביים (this.number == other.number). מכיוון שהמספרים שונים, חלק זה יחזיר false. בנקודה זו, מערכת הריצה קובעת מייד שכל הביטוי הוא שקרי, ועוצרת את הריצה . הבדיקות הבאות—השוואת היתרה והקריאה הכבדה למתודת ה-isEqual של אובייקט הבעלים הפנימי—ייחסכו לחלוטין ולא יתבצעו, מה שחוסך משאבי מערכת יקרים .

נושא 4: ניהול מערך אובייקטים במחלקה מחזיקה

1. אם ננסה להכניס איבר מעבר לגודל המערך הסטטי ללא בדיקה, התוכנית תנסה לגשת לאינדקס 10 במערך שגודלו 10 (אינדקסים 0 עד 9) ותקרוס מייד עם שגיאת ArrayIndexOutOfBoundsException. הקוד שכתבנו מונע זאת באמצעות התנאי if (count < size) . כאשר count מגיע ל-10, התנאי הופך לשקרי, התוכנית מדלגת על שלב ההשמה, עוברת לבליק ה-else, ומציגה הודעת שגיאה מבוקרת למסך במקום לקרוס .
2. משתנים פרימיטיביים כמו 0 או 1 מייצגים ערכים מספריים קבועים ואינם יכולים לייצג ישות של "כלום" או "חוסר באובייקט" . המתודה getAccount מצהירה על טיפוס החזרה שהוא אובייקט מסוג CashAccount . בשפת Java, הדרך היחידה לסמן שמשתנה מסוג אובייקט אינו מצביע על שום ישות קיימת היא .

באמצעות החזרת המילה השמורה null, המייצגת הפניה ריקה.

3. הפתרון המתוחכם שהועלה בשיעור מציע שבכל פעם שהמערך הסטטי מתמלא לחלוטין והמשתמש מבקש להוסיף חשבון חדש, אנו ניצור מאחורי הקלעים מערך חדש וגדול יותר (למשל, כפול בגודלו). לאחר מכן, נעתיק כלולאה את כל האובייקטים מהמערך הישן והמלא אל המערך החדש והרחב, נעדכן את שדה ה-size של הבנק, ונמשיך לעבוד עם המערך החדש שיש בו כעת מקום פנוי. (בשיעור צוין ש-Java יכולת מחלקה מובנית בשם ArrayList שמבצעת מנגנון דינמי זה באופן אוטומטי).